

Web Programming

Degree: Master in Computer Science

Level: M1 / M2
Semester: 2

Total workload:

- ▶ 10 lectures
- ▶ 10 practices
- ▶ CC
- ▶ Exam

Dr. Alla Jammine
alla.jammine@u-paris.fr

1. Course Objectives

At the end of this course unit, students will be able to:

understand the architecture of modern web applications;

design and develop full-stack web applications;

master key technologies used in contemporary web development;

interact with relational and NoSQL databases;

apply best practices in development, security, and deployment;

integrate efficiently into a professional environment (internship or work-study program)

2. Learning Outcomes

Upon successful completion of the course, students will be able to:

structure web interfaces using semantic HTML5;

build responsive user interfaces with CSS3;

program client-side logic using modern JavaScript (ES6+);

develop front-end applications using React;

design RESTful APIs using Django or Node.js/Express;

manipulate SQL databases (MySQL, SQLite) and NoSQL databases (MongoDB);

connect front-end applications to back-end services via APIs;

prepare and deploy web applications in a production environment.

3. Targeted Skills and Competencies

Technical Skills

- ▶ Full-stack web application development
- ▶ REST API design and implementation
- ▶ Database modeling and management
- ▶ Use of modern front-end frameworks (React)

Transversal Skills

- ▶ Technical problem analysis and resolution
- ▶ Teamwork and collaborative development
- ▶ Reading and understanding technical documentation
- ▶ Application of software engineering best practices

4. Prerequisites



Basic programming skills (Python, Java, or equivalent)



Fundamental knowledge of algorithms



Basic understanding of computer systems

Course Plan (10 Sessions)

1. Introduction to Web Development
2. HTML5: Structuring the Web
3. CSS3 & Responsive Design
4. JavaScript Fundamentals
5. Advanced Front-End with React
6. Front-End / Back-End Communication
7. Backend with Django
8. Backend with Node.js / Express
9. Full-Stack Integration
10. Deployment & Project Presentation

Tools and Technologies

Languages: HTML, CSS, JavaScript, Python

Frameworks: React, Django, Express

Databases: MySQL, SQLite, MongoDB

Tools: Git, GitHub, VS Code

Career Prospects and Professional Integration

This course prepares students for roles such as:

- ▶ Web Developer
- ▶ Full-Stack Developer
- ▶ Back-End Developer
- ▶ Junior Software Engineer



Lecture 1 – Introduction to Web Development

Plan:

- ▶ Web development overview: front-end, back-end, databases
- ▶ Client-server architecture
- ▶ HTTP / HTTPS & request-response model
- ▶ REST principles
- ▶ Course organization

Learning Outcome:

Students understand **web architecture**, how client and server interact, and the goals of the course.

What Is Web Development?

Web development is the creation of applications that run on the Web

Modern web applications are **distributed systems**

Based on a **client-server architecture**

Used in business, research, and everyday digital services

Modern Web Technology Stack



Front-end: HTML, CSS, JavaScript, React



Back-end: Django or Node.js / Express



Databases: MySQL, SQLite, MongoDB



Focus on scalability, security, and maintainability

Client-Server Architecture: Overview

Key concepts

- ▶ Client-server architecture is a **distributed system model**
- ▶ Responsibilities are **clearly separated**
- ▶ Multiple clients can interact with one or more servers
- ▶ Communication occurs over a network (Internet)

Definition

A **client** requests services or resources.

A **server** processes requests and provides responses.

Typical examples

- ▶ Web browser ↔ Web server
- ▶ Mobile app ↔ API server
- ▶ Front-end application ↔ Backend service

[Client] <---- request / response ----> [Server]

Client Side (Front-End)

Role of the Client

Runs on the user's device (browser or mobile app)

Responsible for:

- ▶ User Interface (UI)
- ▶ User interactions (clicks, forms, navigation)
- ▶ Sending requests to the server
- ▶ Displaying responses

Technologies used

- ▶ HTML — structure
- ▶ CSS — layout and design
- ▶ JavaScript — logic and interaction
- ▶ Frameworks: **React**, **Vue**, **Angular**

Important notes

- ▶ The client **does not store sensitive logic**
- ▶ The client **cannot be trusted** for security

The frontend is what your users see and includes visual elements like buttons, checkboxes, graphics, and text messages. It allows your users to interact with your application.

Client-Side Frameworks

React: A JavaScript library for building user interfaces, maintained by Facebook. Allows for the creation of reusable UI components and provides a virtual DOM for efficient rendering.

Angular: A platform and framework for building single-page client applications using HTML and TypeScript. Developed and maintained by Google.

Vue.js: A progressive JavaScript framework for building user interfaces. Designed to be incrementally adoptable, with a core library focused on the view layer.

Svelte: A radical new approach to building user interfaces, shifting much of the work to compile time. Produces highly optimized vanilla JavaScript at build time.

Bootstrap: A popular front-end framework for developing responsive and mobile-first websites. Provides pre-designed components and utilities.

Server Side (Back-End)

Role of the Server

Runs on a remote machine (cloud or data center)

Responsible for:

- ▶ Business logic
- ▶ Authentication & authorization
- ▶ Data validation
- ▶ Database access
- ▶ API responses

Technologies used

- ▶ Python (Django, Flask)
- ▶ Node.js (Express)
- ▶ Java, C#, PHP (industry context)

Key characteristics

- ▶ Server controls **security**
- ▶ Server is **stateless** in REST architecture
- ▶ Can serve **many clients simultaneously**

The backend is the data and infrastructure that make your application work.

Server-Side Frameworks

Node.js: A JavaScript runtime built on Chrome's V8 engine, ideal for building scalable network applications.

Express.js: A minimal and flexible Node.js web application framework that provides a robust set of features for web and mobile applications.

Django: A high-level Python web framework that encourages rapid development and clean, pragmatic design. Comes with a built-in ORM, admin interface, and authentication mechanisms.

Spring Boot: A Java-based framework that simplifies the development of new Spring applications. Provides embedded servers and simplifies dependency management.

Ruby on Rails: A server-side web application framework written in Ruby. Emphasizes convention over configuration and the use of the MVC pattern.

What are web servers used for?

Web servers are primarily used to process and manage HTTP/HTTPS requests and responses from the client system.

A web server can also perform several other functions, such as:

- ▶ **Store and protect website data:** A web server can store and protect critical website data from unauthorized users.
- ▶ **Control bandwidth to regulate network traffic:** A web server can help eliminate the downtime caused by high web traffic. Web hosts can set bandwidth to manage the rate of data transmission over the internet and minimize the excess network traffic.
- ▶ **Server-side web scripting:** The server-side web scripting feature enables users to create dynamic web pages using scripting languages such as Ruby, Python, and PHP.
- ▶ **Virtual hosting:** Web servers can also be used as virtual servers to run multiple applications, websites, data, and other services.

How web servers work

The end user processes a request via a web browser installed on a web server. The communication between a web server or browser and the end user takes place using Hypertext Transfer Protocol (HTTP).

The primary role of a web server is to store, process, and deliver requested information or webpages to end users.

It uses:

Physical Storage: All website data is stored on a physical web server to ensure its safety. When an end user enters the URL of your website or searches it using a keyword on a browser, a request is generated and sent to the web server to process the data.

Web browser: The role of web browsers such as Firefox, Chrome, or Internet Explorer is to find the web server on which your website data is located. Once the browser finds your server, it reads the request and processes the information.

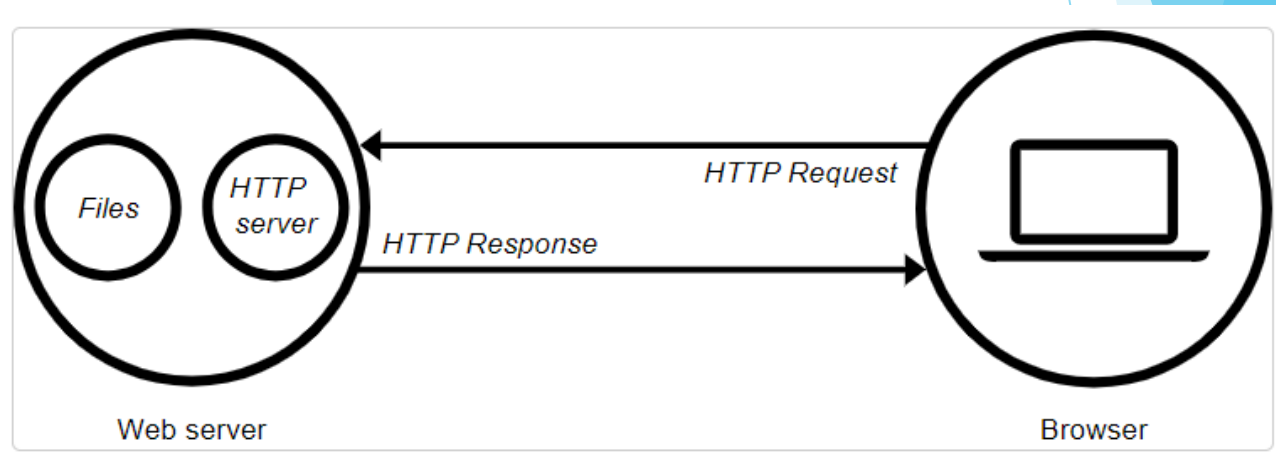
Communication & Request-Response Cycle

How client and server communicate

- ▶ Using HTTP / HTTPS
- ▶ Based on request-response model
- ▶ Often follows REST principles

Typical request flow

- ▶ Client sends HTTP request (GET / POST / PUT / DELETE)
- ▶ Server processes the request
- ▶ Server queries or updates database
- ▶ Server sends HTTP response
- ▶ Client updates the UI



- ▶ **HTTP = HyperText Transfer Protocol**
- ▶ Standard protocol for client-server communication
- ▶ Works on a **request-response** model
- ▶ Uses **methods**: GET, POST, PUT, DELETE
- ▶ Stateless: each request is independent
- ▶ **Diagram (simple):**
- ▶ [Client] -- HTTP Request --> [Server]
- ▶ [Server] -- HTTP Response --> [Client]

HTTP: The Web Protocol

HTTPS = HTTP + TLS (encryption)

Provides:

- ▶ **Confidentiality** (data encrypted)
- ▶ **Integrity** (data cannot be tampered)
- ▶ **Authentication** (server identity verified)

Standard for all modern web applications

Diagram:

[Client] -- HTTPS Request (encrypted) --> [Server]

[Server] -- HTTPS Response (encrypted) --> [Client]

TSL - Transport Layer Security - is a widely adopted security protocol designed to facilitate privacy and data security for communications over the Internet. A primary use case of TLS is encrypting the communication between web applications and servers, such as web browsers loading a website.

HTTPS: Secure HTTP

REST Principles (Overview)

REST = **Representational State Transfer**

Architectural style for web APIs

Core principles:

- ▶ **Resource-based:** each entity has a URL
- ▶ **Stateless:** server does not store client session
- ▶ **Uniform interface:** standard HTTP methods

Supports scalability, simplicity, and interoperability

Diagram

[Client] <--> [Server API] <--> [Database]

Resource URLs: /users, /products

HTTP Methods & REST Actions

Key Points:

HTTP Method	REST Action	Example
GET	Retrieve data	GET /users
POST	Create data	POST /users
PUT / PATCH	Update data	PUT /users/123
DELETE	Delete data	DELETE /users/123

Responses include **status codes**:

- 200 OK, 201 Created, 404 Not Found, 500 Server Error

Client --> GET /users --> Server --> 200 OK + JSON --> Client

What Is a Technology Stack?

A **technology stack** is a combination of tools, frameworks, and technologies used to build a web application.

- ▶ Divided into **Front-end**, **Back-end**, and **Database** layers.
- ▶ Choosing the right stack affects **performance**, **scalability**, and **maintainability**.

[Front-end] <--> [Back-end] <--> [Database]

Front-End Technologies

Runs in the **user's browser**. Responsible for **UI and interaction**.

Core technologies:

- ▶ **HTML**: structure and content
- ▶ **CSS**: layout, design, styling
- ▶ **JavaScript**: behavior, interactivity

Modern frameworks: **React** (most popular), Vue, Angular

Optional libraries: Bootstrap, Tailwind for styling

[User Browser]

- HTML
- CSS
- JS
- React Components

Back-End Technologies

Runs on a **server**, responsible for **business logic**, **authentication**, **API endpoints**, and **data processing**.

Common frameworks:

- ▶ **Python / Django**: full-featured, rapid development
- ▶ **Node.js / Express**: JavaScript backend, lightweight and scalable

Other options: Java (Spring), PHP, C# (ASP.NET)

Provides **APIs** for front-end c

[Server]

- API
- Business Logic
- Security
- Interacts with Database

communication

Databases & Integration

Stores and manages data for the application.

Types:

- ▶ **Relational:** MySQL, PostgreSQL, SQLite → structured data, ACID compliance
- ▶ **NoSQL:** MongoDB → flexible schema, scalable for large datasets

Back-end frameworks connect to databases via **ORM (Object-Relational Mapping)** or drivers.

Integration ensures **data consistency, security, and performance.**

[Front-end] <--API--> [Back-end] <--DB Driver/ORM--> [Database]

From Local Development to Public Website

Applications are first developed locally on your computer.



To share them with the world, they must be **hosted** on a **server** with a public address.



Main steps:

Choose hosting

Prepare production build

Deploy and configure domain



[Developer Computer] --deploy--> [Server / Cloud] --> [Users]

Hosting Options



Traditional hosting: shared, VPS (small projects)



Cloud platforms: AWS, Google Cloud, Azure (scalable)

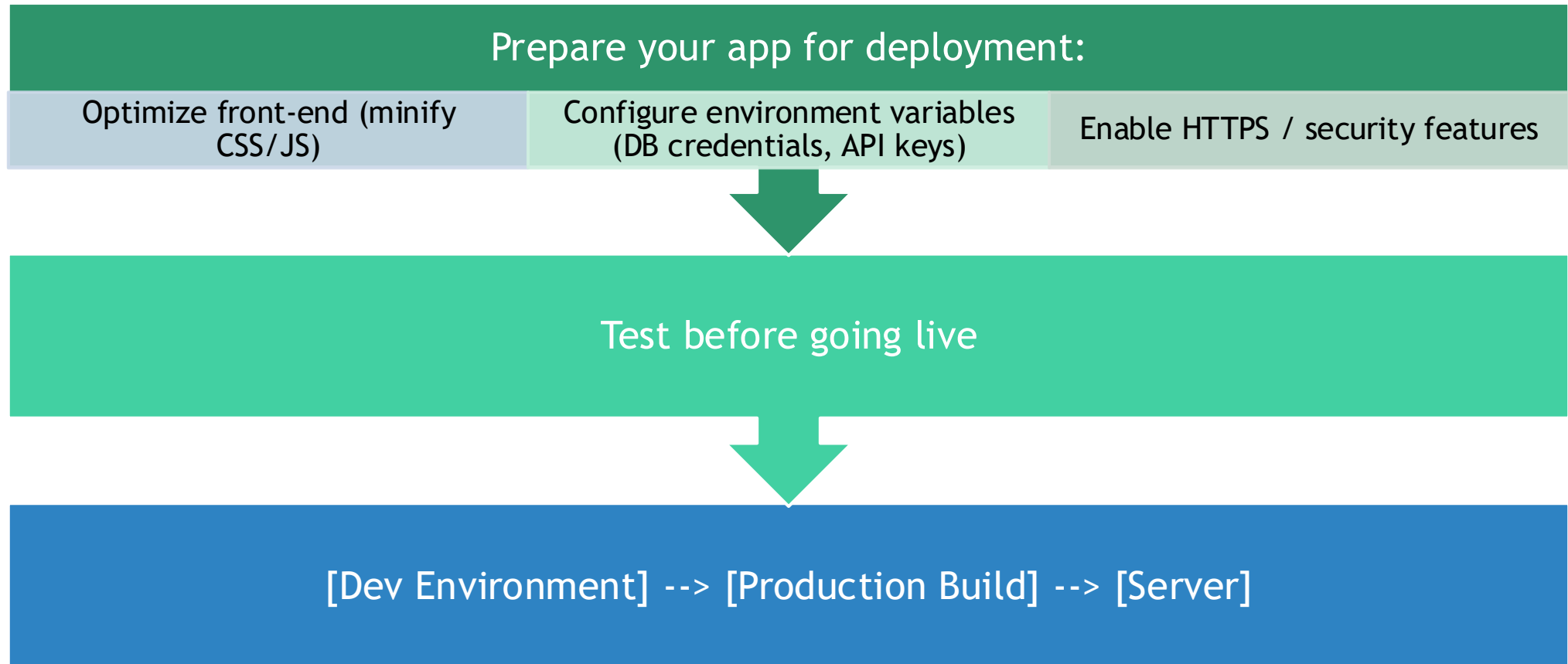


Modern PaaS: Heroku, Vercel, Netlify (easy for web apps)



[Application] --> [Hosting / Cloud] --> [Public Users]

Production Ready



Deploy & Access

Deployment methods:

- ▶ Git push (Heroku, Vercel)
- ▶ FTP/SFTP upload
- ▶ Optional: Docker containers

Configure domain and DNS → users can access via browser

[Deployment Method] --> [Server / Cloud] --> [Domain Name] --> [Users]

Mini-Test (15 Qs)

1. What is the primary role of the client in web development?

- A) Store and manage database
- B) Display UI and handle user interactions
- C) Secure the server
- D) Process server-side business logic

2. Which component processes business logic and communicates with the database?

- A) Client
- B) Server
- C) Browser
- D) HTML

3. HTTP stands for:

- A) HyperText Transfer Protocol
- B) Hyper Tool Text Processor
- C) High Transfer Text Protocol
- D) HyperText Transfer Page

4. HTTPS is different from HTTP because it:

- A) Uses JSON
- B) Encrypts communication
- C) Does not use TCP
- D) Runs only on local computers

5. What does REST stand for?

- A) Representational State Transfer
- B) Rapid Server Transfer
- C) Relational Structured Template
- D) Resource Standard Technique

6. Which of the following is NOT a REST principle?

- A) Resource-based
- B) Stateless communication
- C) Uniform interface
- D) Client stores session state

```
mirror_mod = modifier_ob.  
# Add mirror object to mirror  
mirror_mod.mirror_object  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True  
  
# Selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
  
print("please select exactly  
  
--- OPERATOR CLASSES ---  
  
types.Operator):  
on X mirror to the selected  
object.mirror_mirror_x"  
mirror X"  
  
context):  
context.active_object is not
```

7. Which HTTP method is used to retrieve data?

- A) POST
- B) GET
- C) PUT
- D) DELETE

8. Which front-end technologies are used to create user interfaces?

- A) Django and Node.js
- B) HTML, CSS, JavaScript
- C) MySQL, MongoDB
- D) Git, GitHub

9. Which framework is commonly used for building modern front-end SPAs?

- A) React
- B) Django
- C) Express
- D) MongoDB

```
mirror_mod = modifier_ob.  
#set mirror object to mirror  
mirror_mod.mirror_object  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True  
  
#selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
  
print("please select exactly  
  
-- OPERATOR CLASSES -----  
  
types.Operator):  
on X mirror to the selected  
object.mirror_mirror_x"  
mirror X"  
  
context):  
context.active_object is not
```

10. What is a technology stack?

- A) A collection of programming languages only
- B) The combination of front-end, back-end, and database technologies
- C) A single software framework
- D) Only the server-side software

11. Which of the following is a relational database?

- A) MongoDB
- B) MySQL
- C) Redis
- D) Firebase

12. Which database type offers flexible schema and scalability for unstructured data?

- A) Relational database
- B) NoSQL database
- C) CSV file
- D) SQLite


```
mirror_mod = modifier_ob.  
#set mirror object to mirror  
mirror_mod.mirror_object  
operation == "MIRROR_X":  
mirror_mod.use_x = True  
mirror_mod.use_y = False  
mirror_mod.use_z = False  
operation == "MIRROR_Y":  
mirror_mod.use_x = False  
mirror_mod.use_y = True  
mirror_mod.use_z = False  
operation == "MIRROR_Z":  
mirror_mod.use_x = False  
mirror_mod.use_y = False  
mirror_mod.use_z = True  
  
#selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
  
print("please select exactly  
  
-- OPERATOR CLASSES ----  
  
types.Operator):  
on X mirror to the selected  
object.mirror_mirror_x"  
mirror X"  
  
context):  
context.active_object is not
```

13. What is the main purpose of deploying a web application?

- A) To test locally
- B) To make it publicly accessible
- C) To compress the code
- D) To write documentation

14. Which of these is a common PaaS for web app deployment?

- A) Netlify
- B) MySQL
- C) HTML
- D) Git

15. Which of these is true about the request-response cycle?

- A) Client sends request, server sends response
- B) Server initiates request to client
- C) Client stores server data permanently
- D) HTTP is used only on local networks

1. B
2. B
3. A
4. B
5. A
6. D
7. B
8. B
9. A
10. B
11. B
12. B
13. B
14. A
15. A

Answer Key



Thank you!